TiNA: Tiered Network Buffer Architecture for Fast Networking in Chiplet-based CPUs

Siddharth Agarwal*
University of Illinois Urbana-Champaign
Urbana, USA
sa10@illinois.edu

Tianchen Wang*
University of Illinois Urbana-Champaign
Urbana, USA
tw12@illinois.edu

Jinghan Huang
University of Illinois Urbana-Champaign
Urbana, USA
jinghan4@illinois.edu

Saksham Agarwal
University of Illinois Urbana-Champaign
Urbana, USA
saksham@illinois.edu

Nam Sung Kim
University of Illinois Urbana-Champaign
Urbana, USA
nskim@illinois.edu

Abstract

To manufacture a large CPU cost-effectively, the industry has begun exploiting emerging packaging technologies that integrate multiple chiplets—each comprising a subset of cores and/or memory and I/O subsystems—into a single package. However, such a CPU experiences longer memory access latency with more pronounced variance, especially when its cores in one chiplet access LLC slices[1] or DRAM controllers in other chiplets. This creates unique challenges in µs-scale networking, which is highly sensitive to memory access latency. In this work, we start by proposing exploiting a little-known mode, known as Sub-NUMA Clustering (SNC), in the latest chiplet-based CPUs. As it restricts receiving and processing packets to a particular chiplet unless explicitly specified otherwise, it offers shorter memory access latency and, consequently, lower networking latency than the default mode (non-SNC). Nonetheless, when receiving long bursts of packets[2], SNC incurs higher networking latency than non-SNC, as it provides less LLC capacity for CPU cores processing the packets, making Direct Cache Access (DCA)—a commonly used CPU feature to reduce memory access latency for packet processing—ineffective. To address this drawback, we propose TiNA, a tiered network buffer architecture consisting of an enhanced NIC and networking stack, which opportunistically uses LLC slices in other chiplets for DCA only when receiving long bursts of packets. On average, TiNA reduces the mean (tail) latency by 25% (18%) and 28% (22%), compared to SNC and non-SNC, respectively, across diverse network applications and traces.

CCS Concepts: • **Computer systems organization → Architectures**; • **Networks**;

Keywords: Sub-NUMA Clustering, Direct Cache Access, Chiplet, Direct Memory Access, Non Uniform Cache Access

ACM Reference Format:
Siddharth Agarwal, Tianchen Wang, Jinghan Huang, Saksham Agarwal, and Nam Sung Kim. 2026. TiNA: Tiered Network Buffer Architecture for Fast Networking in Chiplet-based CPUs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3760250.3762224

1 Introduction

Increasing the number of cores per CPU reduces the Total Cost of Ownership (TCO) for hyperscalers [25]. Shared resources like the Last-Level Cache (LLC), DRAM controllers and I/O lanes need to scale up proportionally with the number of CPU cores, which necessitates a larger chip as Moore's Law has come to an end. However, this trend towards a larger chip has been hindered by the lithographic reticle size and yield [29]. To address this challenge, the industry has turned to emerging packaging technologies, such as Embedded Multi-die Interconnect Bridge (EMIB) [30] and Elevated Fanout Bridge 2.5D [29], where multiple chiplets are placed on a package substrate and connected with on-package interconnects. For example, the Intel® Xeon® Scalable Processor—codenamed Sapphire Rapids (SPR) [17]—consists of up to 4 chiplets, each integrating a full set of CPU cores, LLC slices,

*Both authors contributed equally to this research.
[1]A slice is a subset of the LLC directly connected with a CPU core. The CPU core can access the LLC slices of other CPU cores through interconnects but at longer latency than its own LLC slice.
[2]A burst is defined as the period during which the NIC receives packets at line rate, and bursts lasting hundreds of microseconds frequently occur in datacenter network traffic [43]

ASPLOS '26, Pittsburgh, PA, USA

ACM ISBN 979-8-4007-2165-6/26/03
https://doi.org/10.1145/3760250.3762224

DRAM controllers, and I/O lanes. That is, a single chiplet is a small but complete CPU.

By default, these chiplets operate together as a single large CPU, where each CPU core in a chiplet can access all LLC slices and DRAM controllers in the CPU in an interleaved fashion. While the on-package interconnect is notably faster than the cross-socket link, such as Ultra Path Interconnect (UPI) [16], it still introduces tens of nanoseconds of latency, which is several times slower than the on-chip interconnect. As memory accesses cross the on-package interconnect multiple times depending on their addresses, they experience higher LLC and DRAM access latency with greater variance, which can notably affect the end-to-end latency of µs-scale networking, as processing a packet requires multiple memory accesses. Alternatively, we can exploit a little-known mode in the SPR CPU, known as Sub-NUMA Clustering (SNC), which logically divides the CPU at chiplet boundaries [17]. It makes each chiplet recognized as a unique NUMA node by the operating system (OS), providing strong isolation of CPU resources—CPU cores, LLC slices, DRAM controllers, and I/O lanes—among chiplets (or sub-NUMA nodes). As it restricts memory accesses to the local chiplet unless memory regions are explicitly allocated to other SNC nodes, it offers lower memory access latency, but at the cost of reducing LLC capacity and DRAM bandwidth to a quarter.

In this work, we demonstrate the benefit and drawback of SNC over non-SNC for µs-scale networking, and propose TiNA, a **Ti**ered **N**etwork buffer **A**rchitecture to address SNC's drawback while preserving its benefit. Our specific contributions are as follows.

Contribution 1: Characterization of the Networking Performance of a Chiplet-based CPU. As µs-scale networking performance strongly depends on memory access performance, we begin by comparing the memory access performance between SNC and non-SNC. Subsequently, using a networking benchmark based on Data Plane Development Kit (DPDK) [19]—a kernel bypass networking stack [12] typically used for —we compare packet processing latency between SNC and non-SNC under various burst lengths common in datacenters [43]. We find that SNC can decrease the 50^{th}-percentile (p50) and 99^{th}-percentile (p99) packet processing latency by up to 45% and 50%, respectively, compared to non-SNC, but only for burst lengths shorter than 400 µs. For longer burst lengths, however, SNC can increase the p50 and p99 packet processing latency by up to 20% and 50%, respectively. This is because, with only a quarter of the full LLC capacity for CPU cores processing the packets, Direct Cache Access (DCA)—a commonly used CPU feature to reduce memory access latency for packet processing by directly DMA-writing received packets to specific LLC ways (referred to as DCA ways henceforth) instead of DRAM—suffers from exacerbated DMA leaks [9, 37, 41] and DMA bloats [2, 38], becoming ineffective.

Contribution 2: Tiered Network Buffer Architecture and Prototyping. To address SNC's drawback while preserving its benefit for packet processing, we propose TiNA, which consists of an enhanced NIC and networking stack, referred to as TiNA-NIC and TiNA-stack, respectively. TiNA-stack begins by setting up `Local-tier` and `Remote-tier`, which are network buffers—where the NIC DMA-writes received packets—backed by memory regions in local and remote chiplets relative to the processing core, respectively, and cached in DCA ways within their corresponding chiplets. Upon receiving packets, TiNA-NIC decides whether to place them in `Local-tier` or `Remote-tier` based on the measured size of the received packets and the estimated available capacity of their respective DCA ways, with the latter assisted by TiNA-stack. It then exploits the Receiver-Side Scaling (RSS) capability to DMA-write these packets to specific DCA ways caching the chosen tier. On average, TiNA reduces mean (p99) latency by 25% (18%) and 28% (22%), compared to SNC and non-SNC, respectively, across diverse network applications and traces.

2 Background

In this section, we introduce a chiplet-based CPU architecture, explain interactions between DCA and a NIC, and describe DPDK packet processing models.

2.1 Chiplet-based CPU Architecture

The SPR CPU consists of 4 chiplets, as depicted in Figure 1. For example, a chiplet of an Intel® Xeon® Gold 6414U CPU comprises eight CPU cores, eight 1.875MB LLC slices, two DDR5-4800 DRAM controllers, and 12 PCIe 5.0 lanes, constituting a small but complete CPU. The on-chip cache-coherent interconnect within a chiplet is extended transparently over the on-package interconnect (EMIB) to connect a chiplet to the two neighboring chiplets. When the 4 chiplets operate together as a unified CPU, a CPU core in a chiplet accesses any LLC slices and DRAM controllers across all chiplets. Given memory addresses are mapped to the LLC slices and the DRAM controllers in an interleaved manner, allowing the CPU cores to make use of the capacity of all LLC slices and the bandwidth of all DRAM controllers. In such an architecture, on average, 1 out of 4 memory accesses are quickly served by local LLC slices or DRAM controllers, crossing only the on-chip interconnect within the chiplet (Figure 1-❶). However, the remaining 3 memory accesses cross the on-package interconnect once (Figure 1-❷) or twice (Figure 1-❸), with each crossing taking ~25 ns (5×) longer than crossing the on-chip interconnect, incurring additional latency. In this mode, memory addresses are distributed across chiplets at 256 byte granularity based on Intel's Complex Addressing Scheme [27]. This makes NUCA and NUMA effects more pronounced even within a single socket. Lastly, the

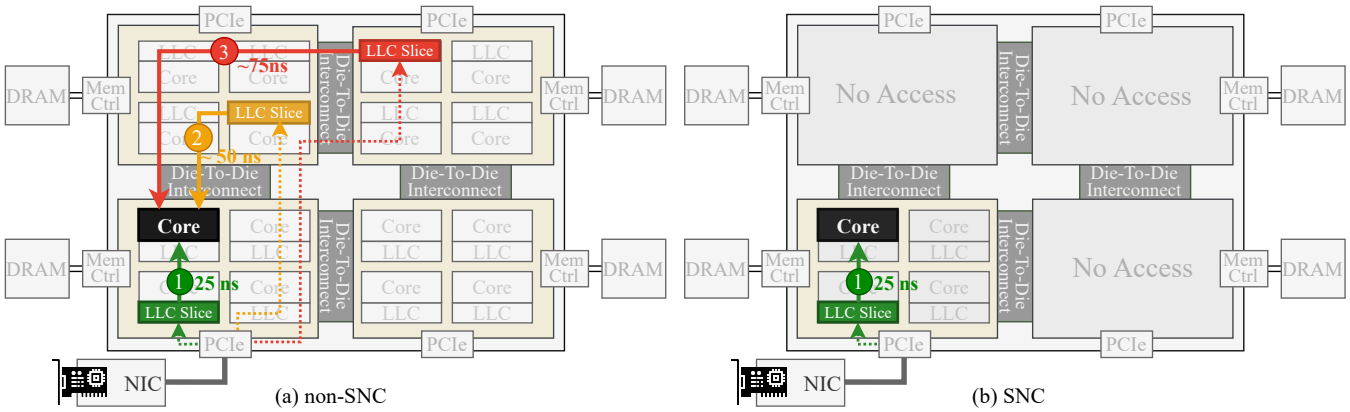


Figure 1. Chiplet-based architecture in Intel® Sapphire Rapids CPU: (a) non-SNC mode with 3 levels of memory access latency and larger LLC capacity and (b) SNC mode with relatively uniform memory access latency and smaller LLC capacity.

latency between a CPU core and an I/O device also depends on the chiplet to which the I/O device is connected.

A single chiplet as a sub-NUMA node. SNC is a little-known mode that allows users to partition resources of a single socket, such as CPU cores, LLC slices, DRAM controllers, and PCIe lanes, into multiple sub-NUMA nodes, each exposed to the OS as a NUMA node [32]. In a chiplet-based CPU architecture, a chiplet can inherently function as a sub-NUMA node. Just like a 4-socket NUMA system, SNC evenly divides the entire memory space into 4 regions, each mapped to the LLC slices and DRAM controllers within each chiplet. SNC, by default, makes a CPU core in a chiplet access only the LLC slices and DRAM controllers within that chiplet unless it is explicitly instructed otherwise (through NUMA aware memory allocation). Therefore, SNC allows a CPU core to access memory notably faster than non-SNC. However, this comes at the cost of reduced LLC capacity and DRAM bandwidth accessible to the CPU core.

2.2 Interaction between DCA and NIC

Traditional DMA path. The classical method to transfer packets between a NIC and a CPU involves DMA. The networking stack allocates a DMA ring buffer for receiving/sending packets to/from the NIC. The NIC increments the tail pointer of the buffer when it receives and DMA-writes a new packet to the buffer, and the CPU core increments the head pointer when it has consumed a packet, allowing the NIC to reuse the corresponding entries in the buffer. These two memory accesses per packet incur long latency and consume high memory bandwidth for packet processing. As network speed has increased to hundreds of Gbps, memory accesses have become a performance bottleneck [1, 15, 40].

DCA-enhanced DMA Path. To reduce the memory bandwidth consumption and memory access latency required for packet processing, Direct Cache Access (DCA) [15] has been implemented in Intel, AMD, and ARM CPUs [4, 20, 26] (*e.g.*, Direct Data I/O (DDIO) for Intel CPUs). DCA allows the NIC to DMA-write packets directly to/from dedicated LLC ways (referred to as DCA ways henceforth). If it finds that buffer entries, to which the NIC will DMA-write packets, are already cached in DCA ways (*i.e.*, on write-hits), it simply write-updates the corresponding cachelines with newly received packets. Otherwise (*i.e.*, on write-misses), it write-allocates the entries to cachelines in the DCA ways after evicting other entries in the cachelines. When the evicted entries store unprocessed packets, DMA leaks occur [9, 37, 41]. This increases both memory bandwidth consumption and packet processing latency, compared to non-DCA. This is because the NIC must wait for the eviction of the entries from the DCA ways before DMA-writing newly received packets to them. Later, CPU cores need to bring these evicted entries from DRAM to process them.

Prior work has identified another challenge with using DCA. DMA bloat [2, 38] , which occurs when cachelines containing already processed packets in Mid-Level Caches (MLCs) are evicted to any LLC ways, contending with co-running application working sets in non-DCA ways and/or newly received packets in DCA ways. The DMA bloats are an artifact of the non-inclusive cache hierarchy in recent CPUs [42], breaking the isolation between DCA and non-DCA ways and/or exacerbating DMA leaks. They can notably degrade the performance of co-running applications, as well as the network application itself, particularly if that application requires additional state (*e.g.*, KVS and NAT) that resides in non-DCA ways.

2.3 DPDK Packet Processing Models

DPDK is a kernel-bypass networking framework [19]. It maintains one or more Rx descriptor ring buffers (or simply descriptor buffers hereafter), where each entry (referred to as a descriptor) stores a pointer to a network buffer (`mbuf`), a memory region (typically 2 kB) to which the NIC DMA-writes a received packet; a descriptor buffer and `mbufs` constitute a DMA ring buffer. `mbufs` are allocated from the

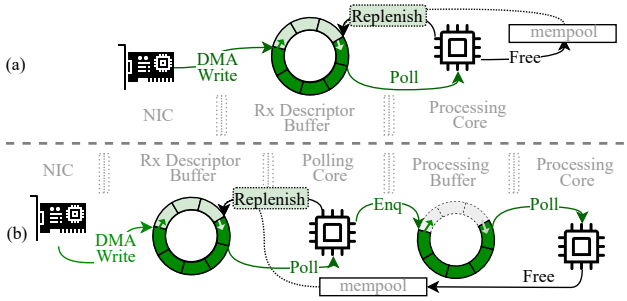


Figure 2. DPDK packet processing models: (a) run-to-completion and (b) pipeline models.

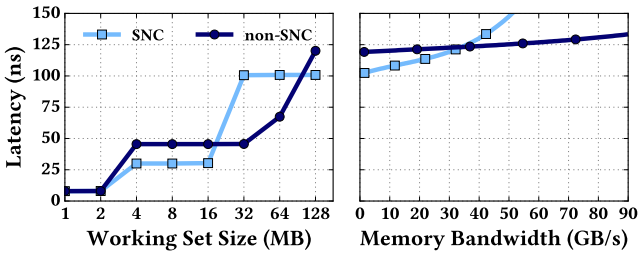


Figure 3. Memory access latency with SNC and non-SNC as (left) the working set size and (right) memory bandwidth consumption are varied, respectively.

`mempool`, a 1 GB huge page region to provide a contiguous memory region for efficient memory access for both the NIC and CPU cores. Through the descriptor buffers, `mbufs`, and `mempool`, DPDK supports two packet processing models: Run-to-Completion (RTC) and pipeline.

RTC Model. When setting up a descriptor buffer with M descriptors, DPDK obtains M `mbufs` in a contiguous memory region from the `mempool`, and statically links a `mbuf` to a descriptor. As depicted in Figure 2(a), a CPU core polls a doorbell register, which stores the tail pointer of the descriptor buffer, to determine whether new packets have been received and written to `mbufs`. Meanwhile, the CPU core processes the packet referenced by the descriptor pointed to by the head pointer and advances the head pointer after it completes processing the packet. As packets are processed and the head pointer advances, `mbufs`, which store previously received and processed packets, are sequentially recycled to store newly received packets. In this model, since the descriptor buffer is directly back-pressured by the CPU core's processing rate, it may run out of unused descriptors when M is not large enough to accommodate a long burst of packets. Therefore, M must be sized to accommodate the longest potential burst of packets to prevent packet drops. However, when the capacity required for storing all `mbufs` with a large descriptor buffer exceeds that of DCA ways and MLC, DMA-writing newly received packets to `mbufs` always causes write-misses in the DCA ways and DMA bloats (§2.2).

Pipeline Model. To address the drawbacks of employing a large descriptor buffer, as shown in Figure 2(b), DPDK sets up a small descriptor buffer along with one or more separate processing buffers. Unlike the RTC model, a CPU core polling the doorbell register simply passes the pointers of `mbufs` storing newly received packets to the processing buffers, instead of directly processing these packets. Concurrently, other CPU cores, each coupled with a processing buffer, poll their respective processing buffers, read the received packets from them, and process them. This model dynamically allocates more `mbufs` if the number of unused descriptors drops below a threshold, and it frees and returns `mbufs` to the `mempool` as soon as packets in these `mbufs` are processed. As it frees and allocates these `mbufs` in a LIFO fashion, it can allocate only a small number of `mbufs`—fitting within DCA ways and MLC—when bursts are short and intermittent [18]. Thus, DMA-writes are likely served by write-hits and do not cause any DMA bloats. Hence, in this work, we use the pipeline model due to its better efficiency for DCA than RTC. However, when bursts are long and/or frequent, this model also suffers from the same drawbacks as the RTC model.

3 Impact of SNC on Performance

In this section, we first evaluate the impact of SNC on memory access performance. Next, we analyze the impact on the networking performance. See Section 6 for a detailed evaluation methodology.

3.1 Memory Access Performance

To evaluate the impact of SNC on memory access performance, we first run a microbenchmark, Intel Memory Latency Checker [39] on an SPR CPU-based server. Figure 3(left) compares the memory access latency between SNC and non-SNC, with the microbenchmark performing pointer chasing for a range of Working set sizes. (W1) 2 MB or smaller: both SNC and non-SNC give the same memory access latency, since the working set fits into the MLC coupled with a CPU core running the microbenchmark. (W2) 2–15 MB: SNC introduces 45% or ~20 ns lower memory access latency than non-SNC since the working set fits into the LLC slices of a chiplet (or SNC node). (W3) 15–60 MB: SNC presents up to 100% longer memory access latency than non-SNC because it provides CPU cores in a chiplet with only a quarter of the full 60 MB LLC capacity, requiring memory accesses to be served by DRAM. (W4) 60 MB or larger: both SNC and non-SNC access DRAM, but SNC offers 20% lower memory access latency than non-SNC, as memory accesses are restricted to the two local DRAM controllers on the same chiplet (§2.1).

Figure 3(right) shows the effect of SNC on DRAM access latency, with the microbenchmark consuming increasingly more DRAM bandwidth. When DRAM bandwidth consumption is low, the memory access latency with SNC and non-SNC is the same as (W4) above. However, as DRAM bandwidth consumption exceeds 30 GB/s, SNC begins to exhibit longer DRAM access latency than non-SNC. This is because

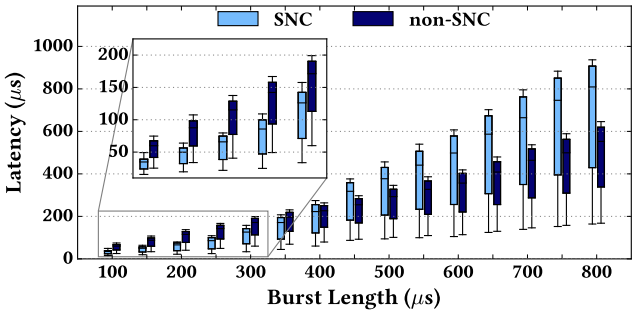


Figure 4. Distribution of packet round-trip latency with SNC and non-SNC for various burst lengths. Each box shows the minimum, p50, p90, p99, and p99.9 latency.

SNC, with only 25% of the DRAM bandwidth of non-SNC (60 GB/s), increases queuing delay at the DRAM controllers more quickly than non-SNC.

3.2 Networking Performance

To evaluate the impact of SNC on networking performance, we run a DPDK-based microbenchmark (`L2TouchForwad`). It is a simple network function that reads the entire 1 kB payload of each received packet, modifies the MAC header field of the packet, and forwards it to the destination. Our test system has 2 MB of DCA ways capacity in SNC mode, and 8 MB in non-SNC. Figure 4 plots the latency distribution of packet round-trip latency with SNC and non-SNC for network traffic with periodic bursts. A burst is defined as the period during which the NIC receives packets at line rate (*i.e.*, 100 Gbps in our setup). We vary the burst lengths from 100 µs to 800 µs. The average link utilization in this experiment, however, is only 10%. Prior work analyzing datacenter network traffic has shown that such µs-scale bursts of packets, with low link utilization outside of these bursts, occur frequently [43]. We observe that SNC gives 60% shorter p50 and p99 latency than non-SNC when the burst length is ~100 µs. However, as the burst length increases, SNC gives nearly the same latency as non-SNC at ~400 µs. Beyond that, SNC starts to present longer latency than non-SNC due to the following reasons.

In the pipeline model (§2.3), the active `mbuf` size (*i.e.*, the number of `mbufs` storing unprocessed packets) dynamically grows in proportion to the burst length. When the burst length is shorter than ~450 µs, SNC increases the active `mbuf` size more slowly than non-SNC since all `mbufs` can be stored in 2 MB of DCA ways, facilitating faster packet processing due to its lower LLC access latency (Figure 5-①). However, as the burst length increases further, the active `mbuf` size exceeds the capacity of DCA ways. This leads to an increase in the number of DMA-write misses, as measured by the 'PCIe Miss' performance counters (Figure 5-②). In this regime, SNC experiences significant DMA leaks, unlike non-SNC, which provides 8 MB of DCA ways. Consequently,

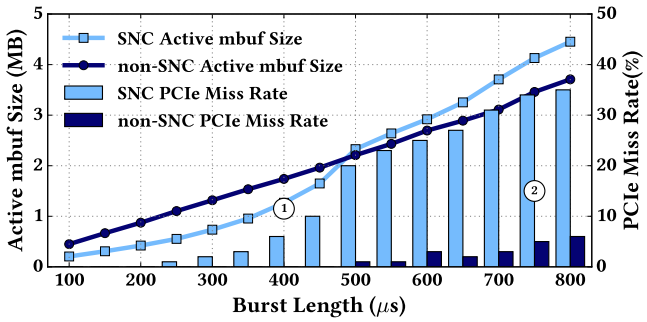


Figure 5. Comparison of the total active `mbuf` size and corresponding PCIe miss rate between SNC and non-SNC.

SNC exhibits progressively longer packet round-trip latency than non-SNC, due to more frequent access to DRAM to retrieve DMA-leaked packets.

Fundamentally, the ideal choice between SNC and non-SNC depends on the size of active `mbufs`. This size depends not only the burst lengths, but also on the processing rate of the network stack and the application, which affects the lifetime of `mbufs` and, consequently, the size of active `mbufs`. While burst length is used here to simplify the illustration of queue buildup, at high loads, several successive short bursts, with small inter burst gap duration can also cause similar queue buildup, as the bursts may arrive before the previous `mbuf` are freed.

We demonstrate this in Figure 6 by varying the packet processing speed of the CPU core. Decreasing the speed shifts the latency break-even point between SNC and non-SNC to a shorter burst of packets due to the longer lifetime of `mbufs`, making it more likely to exceed the capacity of DCA ways in a single chiplet.

4 Tiered Network Buffer Architecture

In this section, to address the drawbacks of SNC for packet processing, we propose TiNA, a tiered network buffer architecture consisting of an enhanced NIC and networking stack, referred to as TiNA-NIC and TiNA-stack. It is designed to maximize the benefit of using SNC—lower core-to-LLC access latency than non-SNC for packet processing—while

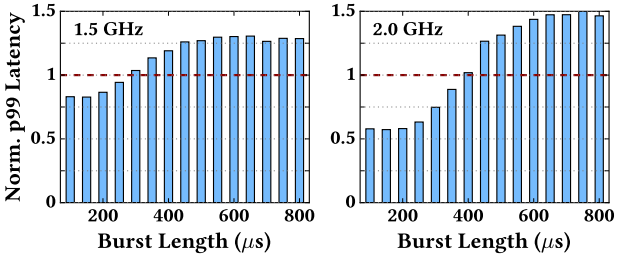


Figure 6. p99 latency of `L2TouchFwd` under SNC normalized to that of non-SNC with the CPU frequency at 1.5 GHz (left) and 2 GHz (right).

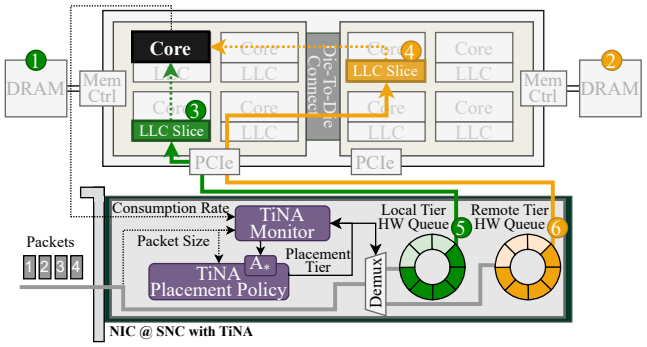


Figure 7. Overview of TiNA with 2 SNC nodes in a CPU.

minimizing its penalty—smaller LLC capacity causing DMA leaks and bloats when active `mbuf` size is large. Specifically, we first overview TiNA. Second, we describe how TiNA-stack enables tiered network buffers. Third, we illustrate how TiNA-NIC implements a packet placement policy that places incoming packets in the appropriate tier based on active `mbuf` size. Finally, we discuss a few cases where the above placement policy could behave sub-optimally, introduce necessary optimizations to handle these cases. We also discuss how TiNA-stack, using its packet processing policies, ensures in-order packet processing.

4.1 Overview

Figure 7 overviews TiNA, consisting of `Local-tier` and `Remote-tier`, where we illustrate a CPU with 2 chiplets for brevity. The first tier, `Local-tier`, is backed by a memory region in the local chiplet (Figure 7-❶), whereas the second tier, `Remote-tier`, is backed by memory regions equally distributed across all $N-1$ remote chiplets (Figure 7-❷), where N denotes the number of chiplets. The use of such a tiered network buffer architecture is motivated by the following key property of the latest chiplet-based CPU architecture: SNC makes memory regions belonging to a chiplet cached only in LLC slices within the same chiplet. This property guarantees that packets DMA-written to `Local-tier` and `Remote-tier` will be cached in DCA ways in the local chiplet and all $N-1$ remote chiplets, referred to as local and remote DCA ways hereafter, respectively. By dynamically using these two tiers, TiNA achieves lower packet processing latency than either SNC or non-SNC alone. For example, during periods when the active `mbuf` size is smaller than the capacity of local DCA ways, TiNA DMA-writes received packets to `Local-tier`, *i.e.*, local DCA ways (Figure 7-❸), offering the benefit of SNC with low core-to-LLC memory access latency through the exclusive use of local DCA ways. Similarly, when the active `mbuf` size exceeds the capacity of local DCA ways, TiNA DMA-write the excess packets to `Remote-tier`, *i.e.*, remote DCA ways (Figure 7-❹), providing the advantage of non-SNC with a larger combined capacity of local and remote DCA ways, obviating DMA leaks.

TiNA-stack allocates N descriptor buffers per packet processing core, mapping one to `Local-tier` in a local chiplet and $N-1$ to `Remote-tier` in remote chiplets, respectively. This allows TiNA-NIC to DMA-write received packets to a specific tier by simply choosing the corresponding descriptor buffer, which is facilitated by its enhanced Receive-Side Scaling (RSS) logic. To minimize packet processing latency, TiNA-NIC implements a dynamic packet placement policy that decides the tier for each received packet before DMA-writing it to the corresponding DCA ways. Since the packet placement decisions depend upon the active `mbuf` size at a given moment, TiNA-NIC implements a hardware-based monitor that continuously measures the size of received packets to estimate the active `mbuf` size. This monitor helps TiNA near-optimally determine whether to DMA-write received packets to `Local-tier` or `Remote-tier`, independent of the workload characteristics such as burst lengths and packet processing rates. TiNA-stack makes sure that packets from different tiers are processed and delivered to a given packet processing application in the same order they were received at TiNA-NIC.

4.2 Enabling Tiered Network Buffers

In contrast to current network stacks employing a single descriptor buffer per packet processing core, TiNA-stack allocates N descriptor buffers per processing core (*i.e.*, one descriptor buffer per chiplet). Descriptors within a descriptor buffer point to `mbufs` backed by the physical memory space of a chiplet to which the descriptor buffer is allocated. The descriptor buffer and its corresponding `mbufs` allocated to the local chiplet compose `Local-tier` cached in local DCA ways. The remaining descriptor buffers and their respective `mbufs`, allocated across all remote chiplets, collectively constitute `Remote-tier` cached in remote DCA ways.

To facilitate the dynamic placement of packets across these `mbufs` in local and remote chiplets, TiNA-NIC builds upon commodity NICs that support multiple hardware queues. They typically employ RSS to (statically) map each hardware queue (Figure 7-❺ and ❻) to a descriptor buffer. RSS determines the hardware queue to enqueue each received packet by using a static but user-programmable hash function with the 5-tuple information in packet header fields as an input [5]. The packets in each hardware queue are DMA-written to `mbufs` pointed to by descriptors of the correspondingly mapped descriptor buffers. As TiNA-NIC uses a dedicated hardware queue for each descriptor buffer, it needs N hardware queues per packet processing core in our setup; commodity NICs—ConnectX-5 and its successors—support more than 512 hardware queues (or 128 processing cores, each with 4 hardware queues in our setup). Note that TiNA organizes these 4 hardware queues into a logical hardware

queue selected by the hash function with the 5-tuple information, and a hardware queue within the logical hardware queue is chosen as described below.

4.3 Packet Placement Policy

Deciding Tiers for Received Packets. Packet processing latency is minimized when a packet resides in local DCA ways at the time of processing §3.2). Otherwise, the latency increases progressively when the packet is placed in remote DCA ways, followed by the local chiplet DRAM, and then the remote chiplet DRAM, in that order. Based on this insight, TiNA implements a packet placement policy using a simple Finite State Machine (FSM) with two states—`local` and `remote`—denoting their respective tiers where TiNA-NIC places received packets. At any given moment, A_{local} and A_{remote} refer to the sizes of active `mbufs` in `Local-tier` and `Remote-tier`, respectively, while `P` refers to the size of received packets for which TiNA is making the placement decision. D_{local} and D_{remote} refer to the capacity of DCA ways in their respective tiers. By default, TiNA starts with `local`, and then switches to `remote` if placing these packets in `Local-tier` would lead to DMA leaks (*i.e.*, $A_{local} + P > D_{local}$). TiNA switches back to `local` when all packets in `Local-tier` first and then in `Remote-tier` are processed. This ensures that packets are processed in the same order in which they are received.

Estimating A_{local} and A_{remote}. The placement decision is made based on A_{local}, A_{remote} and `P`. Since TiNA must decide which hardware queue (*i.e.*, tier) to place packets in immediately after receiving the packets, we choose to implement the packet placement policy within TiNA-NIC. While it can immediately and precisely measure `P` on its own, it requires assistance from TiNA-stack to determine A_{local} and A_{remote}, since it depends on both the rates of receiving and processing packets, and TiNA-NIC cannot determine the latter on its own.

The estimation of A_{local} and A_{remote} within TiNA-NIC involves two key Steps. (S1) For received packets of size `P`, TiNA-NIC makes a placement decision after computing both $A_{local} + P$ and $A_{remote} + P$. Depending on the decision, it increments either A_{local} or A_{remote} by `P`. (S2) TiNA-NIC periodically decrements A_{local} or A_{remote} based on the packet consumption rate. As the packet consumption rate (denoted by `C`) can vary depending on the packet processing application and system, TiNA-stack periodically sends the average value of `C` to TiNA-NIC at every `I` seconds (§5). With given `I`, A_{local} and A_{remote}, TiNA-NIC decides when and which `A` must be decremented. If only A_{local} is nonzero, TiNA-NIC simply decrements it at the consumption rate until it reaches zero. If A_{remote} is nonzero, as TiNA-NIC needs to decrement A_{local} and A_{remote} in the same order as the TiNA-stack processes packets, it first decrements A_{local} at `C` until it reaches zero, and then it decrements A_{remote}.

4.4 Advanced Packet Placement Policy with Processing Ordering Guarantee

The naive packet placement policy (§4.3) transitions to `local` only after all packets in `Local-tier`, and then in `Remote-tier`, have been processed, to guarantee they are processed in the order they were received. However, it is sub-optimal for the following Cases: (C1) $A_{local} = D_{local}$ and $A_{remote} + P > D_{remote}$, which occurs when TiNA-NIC receives a long burst of packets. (C2) $A_{local} < D_{local}$ and $A_{remote} > 0$, as the CPU core has processed packets in `Local-tier` and the corresponding `mbufs` in local DCA ways are freed. To address (C1) and (C2), TiNA introduces two Optimizations for the packet placement policy. (O1) It transitions to `local` to place the packets in `Local-tier`, which provides lower DRAM access latency than `Remote-tier` since it cannot avoid DMA leaks to DRAM for (C1), regardless of whether TiNA-NIC places received packets in either `Local-tier` or `Remote-tier`. (O2) It can opportunistically transition back to `local` to exploit the freed capacity of local DCA ways for (C2).

While (O1) and (O2) help TiNA further reduce packet processing latency, they complicate the packet processing order for the CPU core. Specifically, newly received packets in `Local-tier` must be processed only after all packets in `Remote-tier` have been processed, unlike the naive packet placement policy. To process packets in the correct order, if newly received packets are placed in `Local-tier` before all packets in `Local-tier` and `Remote-tier` are processed, TiNA-stack checks the packet sequence number of the next packet to be processed in each tier (denoted by S_{local} and S_{remote}) and processes the packet with the smaller sequence number first. TiNA exploits the sequence numbers provided by underlying transport protocols, such as TCP, or places its own sequence number if none exists. Section 5.1 introduces optimizations to reduce the overhead of checking every packet across the tiers.

Lastly, considering this packet processing strategy, we further optimize (O2), opportunistic transitioning to `local`

Algorithm 1: Packet placement and processing

```
if PlacementTier is local then
    if A_local + P > D_local then
        PlacementTier ← remote
    else A_local ← A_local + P
else
    if (A_local = D_local ∧ A_remote + P > D_remote) ∨ (A_local + P <
      D_local') then PlacementTier ← local
    else A_remote ← A_remote + P
if ProcessingTier is local then
    if (A_local = 0 ∧ A_remote > 0) ∨ (A_local > 0 ∨ A_remote > 0) ∧
      S_local > S_remote then ProcessingTier ← remote
else
    if (A_remote = 0) ∨ (A_local + A_remote > 0 ∧ S_local < S_remote)
      then ProcessingTier ← local
```

if $A_{local} + P < D_{local}$, (a slightly lower threshold than D_{local} to prevent oscillations between the two states).

Otherwise, if every other packet were placed in a different tier, it would increase the overhead of checking the sequence numbers and reduce batching efficiency, potentially reducing the performance benefit of TiNA. Algorithm 1 holistically describes the policy for placing packets and processing them under (O1) and (O2).

5 Implementation

In this section, we present the TiNA-NIC and TiNA-stack implementations in detail.

5.1 TiNA-NIC

While we envision TiNA-NIC as an enhancement to the RSS logic in commodity NICs, our evaluation prototype leverages a Xilinx U280 FPGA, connected to a ConnectX-6 NIC through an Ethernet cable in a bump-in-the-wire fashion. It requires only ~500 LoC in Verilog and minimal FPGA resources—~1 k LUTs (out of a 1 M total) and ~2 k registers (out of 2 M total).

Compared to commodity NICs, TiNA-NIC (Figure 7-bottom) implements the following additional operations per packet. First, for a received packet, if a given application does not use an underlying reliable transport protocol such as TCP (which already generates sequence numbers in packet headers), TiNA-NIC affixes a sequence number to the packet header. TiNA-stack uses these sequence numbers to guarantee the correct packet processing order for the application. TiNA-NIC also records and tracks the sequence number internally to determine which `A` it needs to decrement by `C` (*c.f.* Algorithm 1). Second, TiNA-NIC determines an appropriate tier for each packet based on `PlacementTier` in Algorithm 1. Subsequently, TiNA-NIC enforces the packet placement decisions by steering packets to the corresponding NIC hardware queue using the enhanced RSS logic (*i.e.*, 'Demux' in Figure 7).

Recall that for each packet processing core, TiNA-NIC uses 1 hardware queue to receive packets in `Local-tier` and $N-1$ hardware queues to receive packets in `Remote-tier`; these N hardware queues constitute a logical hardware queue (§4.2). Employing traditional 5-tuple-based hashing, TiNA-NIC directs packets to a logical hardware queue, as commodity NICs do to a hardware queue. Additionally, TiNA-NIC introduces additional bits used internally to assign the packets to a specific hardware queue within the logical hardware queue. TiNA-NIC uses a single bit—set by the packet placement policy—to direct packets to a hardware queue associated with `Local-tier` or one of $N-1$ hardware queues associated with `Remote-tier`. TiNA-NIC takes $\lceil Log_2(N-1) \rceil$ bits from the sequence numbers of the packets to choose 1 of the $N-1$ hardware queues linked to Remote-tier for packet placement. These bits allow TiNA-NIC to load-balance packets across all remote DCA ways.

Note that TiNA-NIC uses higher order bits (*e.g.*, 5^{th}-7^{th} bits) of the sequence number to direct packets across the $N-1$ hardware queues. This optimization makes sure that a batch of consecutive packets, rather than a single packet, is steered to a hardware queue. This not only improves the CPU core's packet processing efficiency, as a single call to `rte_eth_rx_burst` returns a batch of consecutive packets to the processing core, but also reduces the overhead of TiNA-stack peeking at the sequence numbers (§5.2).

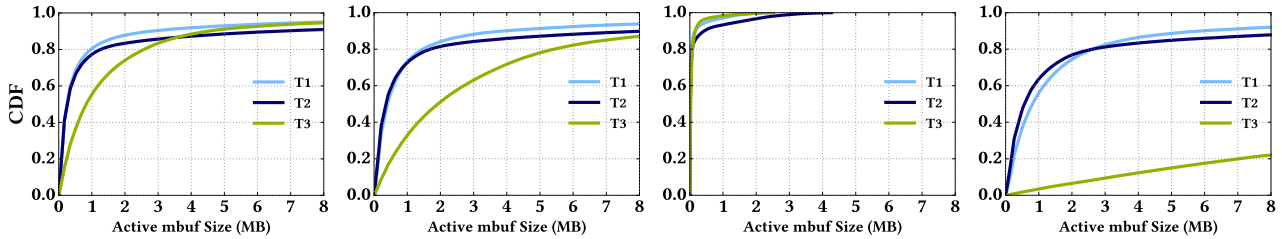
5.2 TiNA-stack

TiNA-stack is responsible for estimating the packet consumption rate (`C`) and sends it to TiNA-NIC every `I` seconds, as well as guaranteeing delivery of packets distributed across multiple descriptor buffers to the application in the correct order. Its implementation requires changes only within the DPDK library, without any changes to the APIs themselves.

Delivering and Processing Packets. TiNA-stack adapts the standard DPDK polling function (`rte_eth_rx_burst`) to poll N descriptor buffers corresponding to the processing core and deliver packets to the application in the correct order as if it were polling a single descriptor buffer. To deliver packets in the correct order, TiNA-stack must peek at the sequence number embedded in the header of the packet at the head of each descriptor buffer. Naively doing so can incur a notable performance overhead from checking the header of the packet at the head of every descriptor buffer. However, TiNA-stack minimizes the overhead by exploiting the packet placement policy (§4.3), with the following Flow.

(F1) TiNA-stack first peeks at the sequence numbers of packets at the heads of these descriptor buffers only when two or more descriptor buffers have packets to be processed, and it remembers (or records) these sequence numbers. (F2) Based on these sequence numbers, TiNA-stack chooses a descriptor buffer with the smallest sequence number, and then processes the packets from it as long as their sequence numbers remain smaller than the recorded sequence numbers from the remaining descriptor buffers. This prevents TiNA-stack from constantly peeking at the sequence numbers of the packets from the remaining descriptor buffers. (F3) While processing packets from the descriptor buffer, if TiNA-stack encounters a larger sequence number than the previously recorded sequence numbers from the remaining descriptor buffers, it records the sequence number, stops processing the packet from the descriptor, and it repeats (F2).

Estimating Packet Consumption Rate. TiNA-stack records the first and second timestamps when the application gets the packet by calling `rte_eth_rx_burst` and returns the `mbuf` to the `mempool` by calling `rte_pktmbuf_free`, respectively. It accumulates the time difference between the two timestamps to a running total. Every `I` seconds, it divides the running total by the total processed bytes to compute the average consumption rate (`C`), and then resets the running total. To reduce the impact of any noise samples for



(a) L2TouchFwd **(b)** KVS **(c)** NAT **(d)** RSA

Figure 8. Distributions of the active mbuf sizes obtained by running L2TouchFwd, NAT, KVS, and RSA.

C, TiNA-stack computes its weighted moving average C_{wma}, using the following formula: $C_{wma} = \frac{1}{8}C + \frac{7}{8}C_{wma}$. It sends C_{wma} to TiNA-NIC through a posted MMIO-write, every I. This has a minimal impact on the processing core's performance since it occurs infrequently, consumes a few hundred cycles (< 400 ns) at most, and handled by a DPDK service core [31]. Note that I is a tunable parameter determined by the application and its input, as well as the system state (*e.g.*, frequency scaling). Smaller values of I lead to more accurate estimation of active mbuf size (A_{local} and A_{remote}) on the TiNA-NIC, but significantly increases the CPU overhead. In our evaluation, we observe that I = 100 µs works effectively not only for the microbenchmark but also for three end-to-end networking applications, providing an accurate estimate of active mbuf size and a negligible CPU overhead.

6 Evaluation

6.1 Methodology

Setup. We evaluate TiNA on a server with an Intel Xeon Gold 6414U Processor, 8 DDR5 4800 DRAM modules (one per channel), a Mellanox ConnectX-6 100GbE NIC, and Ubuntu 22.04 LTS. The server is connected to a single client used as a load generator through the bump-in-the-wire FPGA (§5.1).

Applications. We use a set of network applications that represent typical network functions deployed in software middleboxes [7, 23]. In addition to L2TouchFwd that we evaluated in Section 3.2, we evaluate two other end-to-end network functions: Network Address Translation (NAT) [13] and Rivest–Shamir–Adleman (RSA) [36]. These applications represent not only shallow network functions, which process only the header (NAT), but also deep network functions, which operate on the entire payload (L2TouchFwd and RSA). Additionally, we evaluate a Key-Value Store (KVS) [24] to represent client-serving applications. These applications allow us to evaluate TiNA benefits under varying packet processing rates, application working set sizes, and packet payload access characteristics.

Workloads. The client generates network packets and sends them to the server. It uses a load generator that can generate workloads with arbitrary burst lengths and inter-arrival times between successive bursts, both at µs-scale. We use a payload size of 1 kB—one of the two dominant payload sizes in datacenters [6]—for our evaluation; we find that the efficacy of TiNA is not notably sensitive to the payload size since TiNA makes a packet placement decision primarily based on P (the number of received bytes in a burst). Figure 8 plots the distributions of the active mbuf sizes obtained by running L2TouchFwd, NAT, RSA, and KVS, each with 3 traces based on the distributions of burst lengths and inter-burst gaps collected from a hyperscaler [43]. Note that several short bursts with a small gap in between them have the same impact on active mbuf size as a long burst, as the next burst arrives before the mbufs allocated for previous bursts have been freed. Traces 1 and 2 (T1 and T2) correspond to user-facing applications, while Trace 3 (T3) represents a batch job. These distributions show that the active mbuf size can be as large as 20 MB, and the active mbuf sizes exceed the capacity of local DCA ways (2 MB) 20-50% of the time, depending on the application's consumption rate and the trace.

Metrics. We measure per-packet latency, defined as the time elapsed between when the server's NIC first receives a packet and when it sends the response packet back to the client. This captures the time taken by the NIC's internal receive data path, including DMA, any processing done by the host, and the transmit data path. To precisely measure this latency at µs-scale, we use the same FPGA implementing TiNA-NIC to timestamp packets. The traces are run for a fixed period of 1 minute, and we collect samples of 30 M to 120 M packets depending on the average load of the trace.

6.2 Microbenchmark.

Figure 9(top) shows that TiNA achieves similar or lower latency than both SNC and non-SNC modes, for all evaluated burst lengths. In particular, for burst lengths shorter than 400 µs, where SNC outperforms non-SNC, TiNA matches or improves upon the SNC performance. Figure 9(bottom) shows TiNA achieves ~8-10% lower p99 latency than SNC for burst lengths between 250–400 µs by minimizing any possible DMA leak by dynamically utilizing remote DCA ways when the local DCA ways get filled up. For longer burst lengths, where non-SNC outperforms SNC, TiNA matches or improves upon the non-SNC performance. For burst lengths

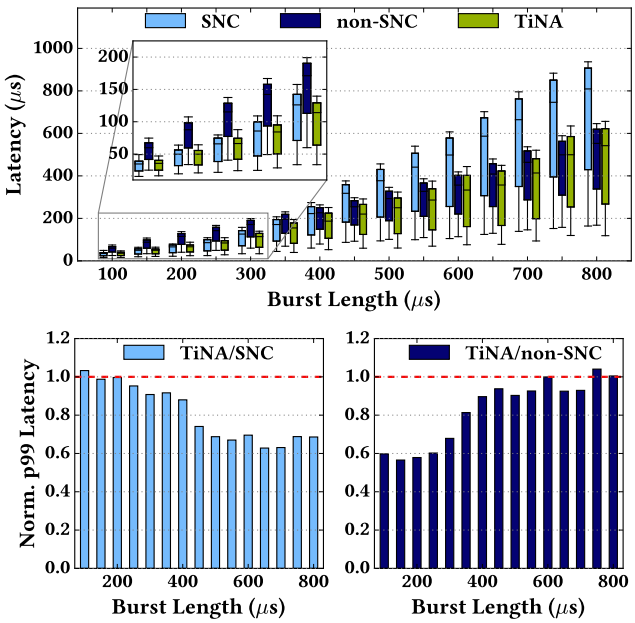


Figure 9. L2TouchFwd latency with TiNA, SNC, and non-SNC (top), and the p99 latency of TiNA, normalized to that of SNC and non-SNC (bottom).

between 400–700 µs, TiNA provides up to 5-10% lower p99 latency by maximizing the use of local DCA ways while preventing DMA leaks. However, TiNA incurs a small latency overhead (≤ 2% for p99 latency) for extremely short and large burst lengths, compared to SNC and non-SNC, respectively. This is because TiNA is unable to provide any additional benefit over SNC and Non-SNC for such extreme burst lengths while incurring a small overhead of polling multiple descriptor buffers (§5.2).

In Figure 10, we plot the p99 latency while increasing the network load up to the maximum value before observing packet loss. This shows that TiNA can provide a better network load-latency tradeoff than both SNC and non-SNC. Like prior work [3, 11, 28], we use a Poisson arrival process to generate network traffic with varying loads—we fix the burst length at 100 µs and sample the inter-arrival times between successive bursts from an exponential distribution (with the mean scaled with respect to average load).

Increasing network load increases the active mbuf size and thus causes SNC to experience DMA leaks earlier than non-SNC. This is evident from the earlier onset of latency inflation and subsequent packet drops, which begin beyond 38 Gbps with SNC, compared to 43 Gbps with non-SNC. This happens because SNC utilizes only the local DCA ways. With DMA leaks slowing down the packet consumption rate, the sustainable network load with SNC is lower than non-SNC. On the other hand, since TiNA also utilizes the remote DCA ways, DMA leaks and packet drops occur as late as non-SNC. Meanwhile, TiNA's ability to prioritize utilizing the local

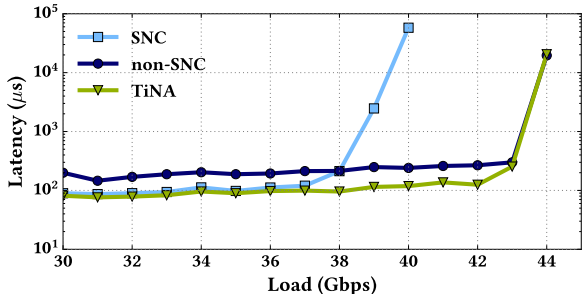


Figure 10. L2TouchFwd p99 latency for different loads.

DCA ways allows it to offer lower p99 latency than non-SNC before unavoidable DMA leaks with very high network loads. In conclusion, TiNA effectively combines the strengths of both SNC and non-SNC for varying workload characteristics, such as burst lengths or network loads.

6.3 End-to-End Applications with Datacenter Network Traces

While Section 6.2 evaluates TiNA for a fixed burst length at each point, this section evaluates TiNA using the three datacenter network traces with varying burst lengths and gaps over time (§6.1). As the maximum active mbuf size can also significantly vary over bursts (Figure 8), both SNC and non-SNC fail to provide the optimal packet processing latency. However, TiNA, with its dynamic packet placement policy, seamlessly benefits from the advantages of both SNC and non-SNC, delivering near-optimal performance regardless of workload characteristics.

Figure 11(top) and (bottom) show improvements in p50 and p99 latency with TiNA when running L2TouchFwd, NAT, KVS and RSA with T1, T2, and T3. For all evaluated applications and traces, TiNA provides similar or lower latency than

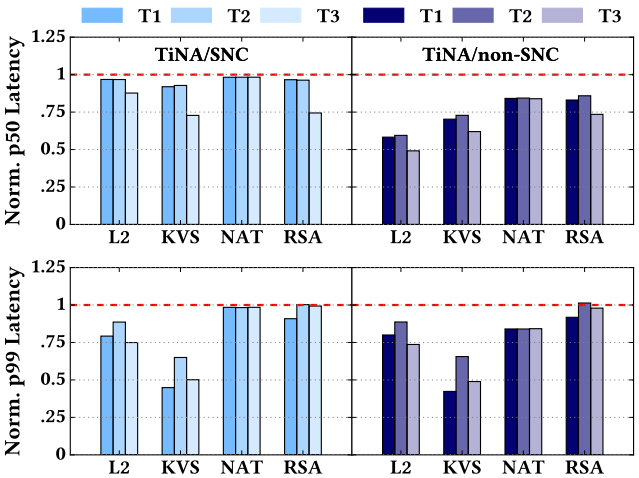
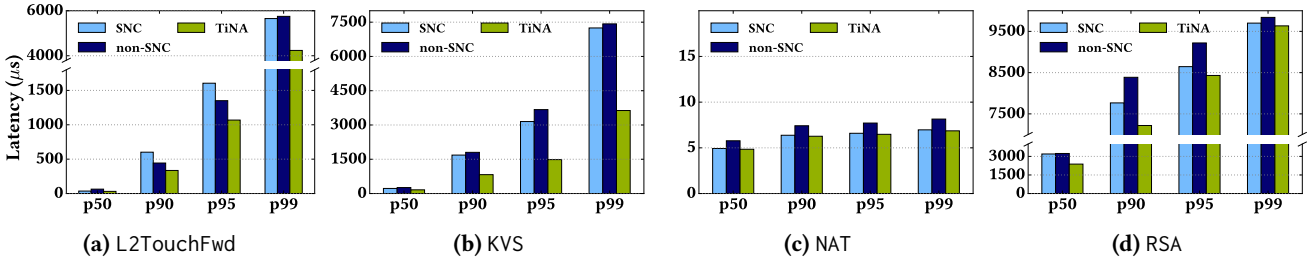


Figure 11. TiNA's p50 (top) and p99 (bottom) network latency normalized to SNC (left) and non-SNC (right), for four different applications running three different traces.



(a) L2TouchFwd **(b)** KVS **(c)** NAT **(d)** RSA

Figure 12. p50, p90, p95 and p99 latencies of evaluated applications running trace T3

both SNC and non-SNC. For example, for L2TouchFwd, we observe 15%–25% lower p99 latency across all traces than both SNC and non-SNC. For KVS we see up to a ~50% reduction in p99 latency. TiNA achieves similar p50 latency to SNC across applications and traces, while improving over non-SNC by up to ~50%. We make the following insightful observations in Figure 11 for different applications, as a result of their respective characteristics:

L2TouchFwd. TiNA leads to ~50% larger improvements in p50 latency over non-SNC compared to SNC. However, it observes similar improvements in p99 latency over both non-SNC and SNC. Investigating it deeper, we uncover that the p50 latency for L2TouchFwd is less than 100 µs (Figure 12a), suggesting that packets experiencing p50 latency arrive when the maximum active mbuf size is smaller than ~700 kB. Figures 4 and 5 demonstrate that SNC provides lower latency when the active mbuf size is smaller than 2MB. Therefore, TiNA performance is closer to SNC (also evident in Figure 12a), providing larger improvements over non-SNC. However, at p99 latency (>4000 µs in Figure 12a), packets arrive with much larger active mbuf sizes (>28 MB). At these sizes, both SNC and non-SNC experience DMA leaks, causing nearly all packets to incur LLC misses when processed by the CPU core. Consequently, both SNC and non-SNC exhibit similar p99 latency, and TiNA provides similar latency improvements over both. TiNA's latency improvements come from the fact that TiNA packet placement policy, during such large active mbuf size, would ensure that packets only in Local-tier would observe (unavoidable) DMA leaks, and the application processing core would incur LLC hits while processing packets in Remote-tier (§4.3).

KVS. TiNA observes the largest gains for p99 latency (up to ~55%) over both SNC and non-SNC for KVS among all evaluated applications. We believe these additional gains in KVS performance are primarily due to TiNA alleviating the DMA bloat problem (§2.2). KVS performance is much more sensitive to DMA bloat than other evaluated applications due to a much larger size of its non-I/O working set, comprising the keys and values for the application. DMA bloat problem occurs when active mbuf size grows larger than the DCA ways, that can later evict cachelines for non-I/O data from LLC. Therefore, KVS benefits from both—the large DCA ways capacity of non-SNC mode, but also the lower core-to-LLC access latency of SNC mode. Hence, TiNA provides its gains by combining the benefits of SNC and non-SNC modes.

NAT. TiNA achieves performance similar to SNC and provides ~20% gains over non-SNC at both p50 and p99. Figure 12c provides further insight: NAT latency remains <10μs even at p99, indicating a very small active mbuf size. This occurs because NAT has a high application consumption rate, as it performs minimal per-packet processing—accessing only a single cacheline per packet. As a result, SNC provides optimal performance even at the tail (shown in Figure 12c). Consequently, TiNA delivers performance close to SNC while improving over non-SNC at both p50 and p99.

RSA. TiNA provides much smaller p99 latency improvements over SNC and non-SNC for RSA than other applications like L2TouchFwd. This is because RSA leads to much larger active mbuf sizes, compared to L2TouchFwd (as evident from our observation of 1.7× larger p99 latency for RSA). As discussed earlier, very large mbuf sizes incur unavoidable DMA leaks and LLC write-misses for almost all packets with both SNC and non-SNC, resulting in similar performance. Further, TiNA's benefits from potentially more LLC write-hits for packets in Remote-tier (see discussion above for L2TouchFwd) get diminished with larger active mbuf sizes. This is because TiNA's packet placement policy now puts even a larger number of packets in Local-tier, which incurs unavoidable DMA leaks. Hence, the fraction of packets placed in Remote-tier avoiding DMA leaks reduces, diminishing overall TiNA benefits.

6.4 Co-Running Non Networking Applications

In data centers, non-networking workloads are often consolidated with networking workloads on the same server. To illustrate the effect of co-running non-networking applications with TiNA, we use two classes of workloads. First, to represent conventional CPU workloads that exert high memory pressure, we select five memory-intensive benchmarks for the SPEC CPU2017 suite [33]. Second, to represent typical datacenter services which can differ significantly in their performance characteristics to traditional benchmarks [22], we run two web-serving workloads from the DCPerf [35] suite. We verified that co-running these non-networking applications has a near-zero impact on the performance of the evaluated networking applications. This behavior was

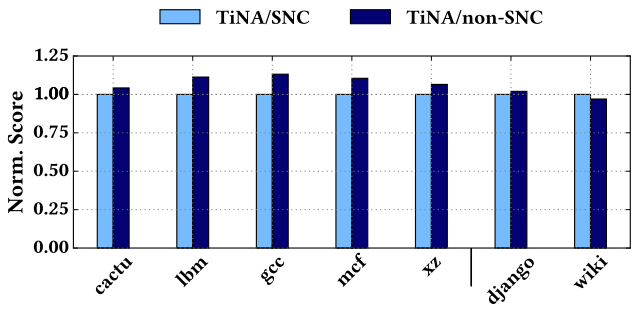


Figure 13. SPEC CPU2017 and DCPerf score while co-running `L2TouchFwd` under TiNA normalized to that of SNC and non-SNC.

expected, due to the isolation of DCA ways for networking traffic via CAT [14].

We also evaluate the impact of using TiNA on the performance of non-networking applications. Figure 13 shows the normalized SPECRate 2017 Integer scores when co-running `L2TouchFwd` with 800 µs bursts. On a 32-core system, we utilize 2 cores for `L2TouchFwd` and run 30 instances of each benchmark, one per remaining core. On average, TiNA matches SNC performance and is about 7% faster than non-SNC for SPEC. These benchmarks benefit from SNC's lower average memory access latency and are not significantly impacted by the reduced LLC capacity. The web-serving benchmarks from DCPerf show little sensitivity to SNC or non-SNC. TiNA's use of DCA ways from `Remote-tier` does not degrade the performance of non-networking applications.

7 Discussion on TiNA Design Decisions

Estimating active `mbuf` size in software. Currently, TiNA-NIC keeps an estimate of the active `mbuf` sizes for both tiers (A_{local} and A_{remote}) in the hardware, to make TiNA's packet placement decisions. Instead, one could also imagine TiNA-stack computing the exact values of A_{local} and A_{remote} by checking the `mempool` and communicating those values periodically to the NIC (*e.g.*, via MMIO writes). However, such a solution can be sub-optimal because of the following reason: TiNA-NIC needs to decide which hardware queue it should place any incoming packet `P` upon immediately receiving `P` at the NIC. However, TiNA-stack can perceive `P` only after it has been DMA-written to the memory from the NIC. Between the NIC-arrival and the corresponding DMA-write, packet `P` can experience queuing delays at NIC receive data path up to several tens of microseconds. Therefore, by the time TiNA-stack could perceive the incoming packets, compute the active `mbuf` sizes and communicate to the NIC, it would be too late for the NIC to make the placement decision.

Using RSS to steer packets to different tiers. Instead of using TiNA's packet placement policy, one could also use unmodified RSS in existing NICs to steer incoming packets between `Local-tier` and `Remote-tier` based on the 5-tuple (which uniquely corresponds to the packet's network flow). Such a packet policy however would result in sub-optimal utilization of DCA ways: with unmodified RSS, the 5-tuple-based mapping remains fixed for the flow's lifetime and does not react to the active `mbuf` size. When active `mbuf` size is low, this is suboptimal because even when there are available DCA ways in `Local-tier`, packets (with flows mapped to the `Remote-tier`) could still be steered to `Remote-tier`. And when the active `mbuf` size is high, this is suboptimal especially in the presence of elephant flows (which are pretty common in datacenter workloads [21]). Specifically, RSS steers all packets from any given elephant flow to the statically-mapped chiplet, potentially suffering from DMA leak even when there might be available DCA ways in other chiplets. In contrast, TiNA adapts to active `mbuf` size, placing packets from all flows in `Local-tier` when DCA capacity permits and even splitting a single flow across `Local-tier` and `Remote-tier` when active `mbuf` size is large.

Using TiNA in non-SNC mode. In this work, we choose SNC as the default operating mode when using TiNA. Alternatively, we can also use TiNA in the non-SNC mode, but it introduces performance overheads. This is mainly because in the non-SNC mode memory accesses are interleaved across chiplets at 256-byte granularity [27]. This necessitates TiNA using a descriptor buffer and associated `mbuf`s spread across non-contiguous memory regions to DMA-write a single packet—when it is larger than 256 bytes—within a single tier. Our evaluation shows that such DMA transactions (also known as split-DMA transactions) incur notable CPU and latency overheads.

8 Related Work

LLC Slice-Aware Memory Allocation. CacheDirector [8] demonstrates the benefits of placing the headers of received packets into the closest LLC slice near the processing core. This is achieved by first reverse engineering the LLC mapping hash function, then dynamically adjusting the `mbuf` headroom to align the packet header with the closest LLC slice. However, unlike TiNA, CacheDirector does not allow dynamically steering packets to DCA ways of other LLC slices to avoid DMA leaks when fully utilizing the closest DCA ways. As CacheDirector achieves finer optimization granularity than TiNA by selecting a specific LLC slice rather than an entire chiplet, it would be interesting to potentially integrate CacheDirector with TiNA to even further reduce memory access latency for specific packet segments.

MLC Prefetching and Invalidation. IDIO [2] utilizes a proactive prefetching technique to move packets from LLC to the processing core's private MLC, before they are explicitly requested by the core. We could potentially integrate such a prefetching technique to further help improve the packet consumption rate in TiNA. Additionally, IDIO and Sweeper [38] proposed a technique to invalidate dirty cachelines storing freed `mbuf`s. This could also be beneficial to

TiNA, as it may reduce DMA bloat, and consequently, contention in the LLC. Nonetheless, these techniques have not yet been implemented in commodity CPUs, while TiNA exploits the available features of the CPUs.

Non Uniform DMA. IOctopus [34] proposed a solution to overcome the penalty of DMA-writes to a NUMA node other than the one to which the NIC is directly connected. It bifurcates the PCIe link of the NIC and connects it to the PCIe root complexes on two NUMA nodes, allowing for direct access to the remote NUMA node's memory and DCA ways. This functionality could be potentially leveraged with TiNA to reduce the DMA-write latency observed by the NIC when DMA-writing to the `Remote-tier`. If the NIC were connected to each Sub-NUMA node's PCIe root complex, it could DMA-write to `Remote-tier` without having to cross the on-package interconnect.

9 Conclusion

In this work, we first characterized the impact of non-uniform cache access latency in multi-chiplet CPUs on µs-scale network applications. We find that naively using the LLC capacity across all chiplets for DCA or restricting it to one chiplet using Sub-NUMA Clustering (SNC) mode leads to sub-optimal latency and throughput for these applications. Based on the insights from our characterization, we then introduced TiNA, a tiered network buffer architecture that provides the benefits of both SNC and non-SNC modes, at runtime. It achieves this by intelligently placing incoming packets to DCA ways in the local or remote chiplets, in response to the changing network traffic characteristics. Our evaluation showed that TiNA significantly improves packet processing latency across a range of network applications and traces, outperforming both SNC and non-SNC.

Acknowledgments

This work was supported in part by generous gift from Intel Corp. and AMD-Xilinx HACC; by the MSIT, Korea, under the Global Scholars Invitation Program (RS-2024-00456287) supervised by the IITP.

A Artifact Appendix

A.1 Abstract

This section describes the resources needed and steps to reproduce the key results presented in the paper. The main artifacts are the TiNA stack layer that runs on DPDK, and the RTL for the TiNA NIC. The artifacts contain scripts to reproduce 1. the motivational results (Fig. 3, 4) and 2. the key evaluation results (Fig. 9, 11)

A.2 Artifact check-list (meta-information)

- **Program:** DPDK-Rx (Runs TiNA stack and benchmarks), DPDK-Tx (Load Generator), Intel MLC [39], Intel PCM
- **Compilation:** GCC 13.0 for software stack and benchmarks, Xilinx Vivado for FPGA bitstream. Corundum [10] repository for Vivado Project and FPGA harness.
- **Binary:** dpdk-rx, dpdk-tx, intel-mlc
- **Run-time environment:** Ubuntu 22.04
- **Hardware:** 4th Gen Xeon (XCC), Xilinx U280, Mellanox ConnectX-6 or later 100 Gbps NIC.
- **Metrics:** p99 latency, throughput
- **Output:** raw text file, figures.
- **Experiments:** Fig. 3, 4, 9 and 12
- **How much disk space required (approximately)?:** ≈ 5 GB (Not including Vivado and other prerequisites)
- **How much time is needed to prepare workflow (approximately)?:** 3 hours (If compiling bitstream)
- **How much time is needed to complete experiments (approximately)?:** 2 hours (Including system restart to reconfigure BIOS)
- **Publicly available?:** https://github.com/ece-fast-lab/ASPLOS-2026-TINA
- **Code licenses (if publicly available)?:** DPDK and Intel PCM are Open Source BSD-3-Clause, Intel MLC is proprietary.
- **Archived: 10.5281/zenodo.16997115**

A.3 Description

A.3.1 How to access.

- The artifact can be accessed on Github at https://github.com/ece-fast-lab/ASPLOS-2026-TINA.
- The artifact repository contains DPDK as a submodule.
- Intel MLC is distributed by Intel
- Intel PCM is distributed by Intel

A.3.2 Hardware dependencies.

- A Sapphire Rapids XCC class CPU, with a 100 Gbps multi-queue NIC(Suggest Mellanox ConnectX6 or later 100 Gbps NIC for script compatibility).
- Xilinx U280
- A second machine capable of generating 100 Gbps traffic, connected to the Sapphire Rapids test machine.

A.3.3 Software dependencies. The main artifact repository contains instructions for installing the dependencies for DPDK and the Python packages required for plotting.

A.3.4 Data sets. The artifact repository contains the generated traces from [43] used in the experiments.

A.4 Installation

First, clone the artifact repository to both the load generator and test machines. Follow the steps in the prerequisites section of the README.MD to install the DPDK library on both the load generator and test machines.

Compile the bit stream following the steps in the FPGA section of the README.MD

A.5 Experiment workflow

The experiments are run using a dedicated script for each experiment on the test, and the load generator machine.

Since the setup requires experiments to be run with Sub NUMA Clustering enabled and disabled, which requires system reboots and BIOS configuration, we recommend first collecting the SNC disabled data for ALL experiments.

Please follow the system manufacturer's guide as the steps will differ for different systems/BIOS versions.

Before experiments are run, we lock the CPU frequency for consistent results. This must be run after every reboot.

```
# NOTE: run these commands on the *test*
cd ~/ASPLOS-2026-TINA
chmod +x flows/prep.sh
sudo  flows/prep.sh
```

We also program and reset the FPGA after building the bitstream.

```
# NOTE: run these commands on the *test*
cd ~/ASPLOS-2026-TINA
cd tina-hw
source /opt/vivado/Vivado/2023.2/settings64.sh
sudo vivado -mode batch -source program_fpga.tcl
```

A.6 Evaluation and expected results

For all experiments, the scripts are in the *flows* folder and their detailed usage, with the exact commands, is described in the README.MD.

A.6.1 Figure 3. The memory experiments are run using Intel MLC. This is the only application that needs only the test machine.

This experiment first sweeps the working set size, and then the memory bandwidth load to produce the working set size versus latency and bandwidth versus latency curves.

Repeat the experiments with SNC enabled.

A.6.2 Figure 4. We first run the baseline dpdk-rx on the test machine and then use the dpdk-tx client program on the load generator machine to generate traffic, which sweeps the burst lengths.

Repeat the experiments with SNC enabled.

A.6.3 Figure 9. For Figure 9, we reuse the data from Figure 4 for baseline numbers, but enable TiNA in the dpdk-rx binary and program the capacity thresholds in the FPGA.

Note that TiNA requires SNC to be enabled.

We use the same dpdk-tx client program to generate the network traffic as before.

A.6.4 Figure 11. The steps to run the end-to-end applications, along with the real-world traces, are similar to the previous experiments. Only the command line arguments to the dpdk-rx binary change to activate the application level processing, and the arguments to the dpdk-tx binary change to replay the recorded traces.

Repeat the experiments with SNC enabled. Note that TiNA requires SNC to be enabled.

References

[1] Saksham Agarwal, Arvind Krishnamurthy, and Rachit Agarwal. 2023. Host Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) *(ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 275–287. https://doi.org/10.1145/3603269.3604878

[2] Mohammad Alian, Siddharth Agarwal, Jongmin Shin, Neel Patel, Yifan Yuan, Daehoon Kim, Ren Wang, and Nam Sung Kim. 2023. IDIO: Network-Driven, Inbound Network Data Orchestration on Server Processors. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture* (Chicago, Illinois, USA) *(MICRO '22)*. IEEE Press, 480–493. https://doi.org/10.1109/MICRO56248.2022.00042

[3] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM* (Hong Kong, China) *(SIGCOMM '13)*. Association for Computing Machinery, New York, NY, USA, 435–446. https://doi.org/10.1145/2486001.2486031

[4] AMD. Accessed in 2025. Smart Data Cache Injection (SDCI) White Paper . https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/58725.pdf.

[5] Tom Barbette, Georgios P. Katsikas, Gerald Q. Maguire, and Dejan Kostić. 2019. RSS++: load and state-aware receive side scaling. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies* (Orlando, Florida) *(CoNEXT '19)*. Association for Computing Machinery, New York, NY, USA, 318–333. https://doi.org/10.1145/3359989.3365412

[6] Theophilus Benson, Ashok Anand, Aditya Akella, and Ming Zhang. 2010. Understanding data center traffic characteristics. *SIGCOMM Comput. Commun. Rev.* 40, 1 (Jan. 2010), 92–99. https://doi.org/10.1145/1672308.1672325

[7] Anat Bremler-Barr, Yotam Harchol, and David Hay. 2016. OpenBox: A Software-Defined Framework for Developing, Deploying, and Managing Network Functions. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) *(SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 511–524. https://doi.org/10.1145/2934872.2934875

[8] Alireza Farshin, Amir Roozbeh, Gerald Q. Maguire, and Dejan Kostić. 2019. Make the Most out of Last Level Cache in Intel Processors. In *Proceedings of the Fourteenth EuroSys Conference 2019* (Dresden, Germany) *(EuroSys '19)*. Association for Computing Machinery, New York, NY, USA, Article 8, 17 pages. https://doi.org/10.1145/3302424.3303977

[9] Alireza Farshin, Amir Roozbeh, Gerald Q Maguire Jr, and Dejan Kostić. 2020. Reexamining direct cache access to optimize {I/O} intensive applications for multi-hundred-gigabit networks. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Virtual, 673–689. https://doi.org/10.5555/3489146.3489192

[10] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen. 2020. Corundum: An Open-Source 100-Gbps Nic. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 38–46. https://doi.org/10.1109/FCCM48280.2020.00015

[11] Peter X. Gao, Akshay Narayan, Gautam Kumar, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. 2015. pHost: distributed near-optimal datacenter transport over commodity network fabric. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies* (Heidelberg, Germany) *(CoNEXT '15)*. Association for Computing Machinery, New York, NY, USA, Article 1, 12 pages. https://doi.org/10.1145/2716281.2836086

[12] Hossein Golestani, Amirhossein Mirhosseini, and Thomas F. Wenisch. 2019. Software Data Planes: You Can't Always Spin to Win. In *Proceedings of the ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) *(SoCC '19)*. Association for Computing Machinery, New York, NY, USA, 337–350. https://doi.org/10.1145/3357223.3362737

[13] Pankaj Gupta, Steven Lin, and Nick McKeown. 1998. Routing lookups in hardware at memory access speeds. In *Proceedings. IEEE INFOCOM'98, the Conference on Computer Communications. Seventeenth Annual Joint Conference of the IEEE Computer and Communications Societies. Gateway to the 21st Century (Cat. No. 98*, Vol. 3. IEEE, IEEE, San Francisco, CA, USA, 1240–1247. https://doi.org/10.1109/INFCOM.1998.662938

[14] Andrew Herdrich, Edwin Verplanke, Priya Autee, Ramesh Illikkal, Chris Gianos, Ronak Singhal, and Ravi Iyer. 2016. Cache QoS: From concept to reality in the Intel® Xeon® processor E5-2600 v3 product family. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 657–668. https://doi.org/10.1109/HPCA.2016.7446102

[15] Ram Huggahalli, Ravi Iyer, and Scott Tetrick. 2005. Direct Cache Access for High Bandwidth Network I/O. In *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA '05)*. IEEE Computer Society, USA, 50–59. https://doi.org/10.1109/ISCA.2005.23

[16] Intel. 2009. An Introduction to the Intel® QuickPath Interconnect — intel.com. https://www.intel.com/content/www/us/en/io/quickpath-technology/quick-path-interconnect-introduction-paper.html

[17] Intel. Accessed in 2025. 4th Gen Intel Xeon Processor Scalable Family, Sapphire Rapids. https://www.intel.com/content/www/us/en/developer/articles/technical/fourth-generation-xeon-scalable-family-overview.html

[18] Intel. Accessed in 2025. Compute Express Link Optimize Memory Usage in Multithreaded Data Plane Development Kit (DPDK) Applications . https://www.intel.com/content/www/us/en/developer/articles/technical/optimize-memory-usage-in-multi-threaded-data-plane-development-kit-dpdk-applications.html.

[19] Intel. Accessed in 2025. Data Plane Development Kit (DPDK). https://www.dpdk.org.

[20] Intel. Accessed in 2025. Intel® Data Direct I/O Technology (Intel® DDIO): Technology Brief. https://www.intel.com/content/www/us/en/io/data-direct-i-o-technology-brief.html.

[21] Srikanth Kandula, Sudipta Sengupta, Albert Greenberg, Parveen Patel, and Ronnie Chaiken. 2009. The nature of data center traffic: measurements & analysis. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement* (Chicago, Illinois, USA) *(IMC '09)*. Association for Computing Machinery, New York, NY, USA, 202–208. https://doi.org/10.1145/1644893.1644918

[22] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. 2015. Profiling a warehouse-scale computer. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture* (Portland, Oregon) *(ISCA '15)*. Association for Computing Machinery, New York, NY, USA, 158–169. https://doi.org/10.1145/2749469.2750392

[23] Georgios P. Katsikas, Tom Barbette, Dejan Kostić, Rebecca Steinert, and Gerald Q. Maguire Jr. 2018. Metron: NFV Service Chains at the True Speed of the Underlying Hardware. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 171–186. https://doi.org/10.5555/3307441.3307457

[24] Hyeontaek Lim, Bin Fan, David G. Andersen, and Michael Kaminsky. 2011. SILT: a memory-efficient, high-performance key-value store. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (Cascais, Portugal) *(SOSP '11)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/2043556.2043558

[25] Pejman Lotfi-Kamran, Boris Grot, Michael Ferdman, Stavros Volos, Onur Kocberber, Javier Picorel, Almutaz Adileh, Djordje Jevdjic, Sachin Idgunji, Emre Ozer, and Babak Falsafi. 2012. Scale-out processors. In *Proceedings of the 39th Annual International Symposium on Computer Architecture* (Portland, Oregon) *(ISCA '12)*. IEEE Computer Society, USA, 500–511.

[26] ARM Ltd. Accessed in 2025. Arm DynamIQ Shared Unit Technical Reference Manual. https://developer.arm.com/documentation/100453/latest/.

[27] Clémentine Maurice, Nicolas Scouarnec, Christoph Neumann, Olivier Heen, and Aurélien Francillon. 2015. Reverse Engineering Intel Last-Level Cache Complex Addressing Using Performance Counters. In *Proceedings of the 18th International Symposium on Research in Attacks, Intrusions, and Defenses - Volume 9404* (Kyoto, Japan) *(RAID 2015)*. Springer-Verlag, Berlin, Heidelberg, 48–65. https://doi.org/10.1007/978-3-319-26362-5_3

[28] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication* (Budapest, Hungary) *(SIGCOMM '18)*. Association for Computing Machinery, New York, NY, USA, 221–235. https://doi.org/10.1145/3230543.3230564

[29] Samuel Naffziger, Kevin Lepak, Milam Paraschou, and Mahesh Subramony. 2020. 2.2 AMD chiplet architecture for high-performance server and desktop products. In *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, IEEE, SAN FRANCISCO MARRIOTT MARQUIS HOTEL, 44–45. https://doi.org/10.1109/ISSCC19947.2020.9063103

[30] Nevine Nassif, Ashley O. Munch, Carleton L. Molnar, Gerald Pasdast, Sitaraman V. Lyer, Zibing Yang, Oscar Mendoza, Mark Huddart, Srikrishnan Venkataraman, Sireesha Kandula, Rafi Marom, Alexandra M. Kern, Bill Bowhill, David R. Mulvihill, Srikanth Nimmagadda, Varma Kalidindi, Jonathan Krause, Mohammad M. Haq, Roopali Sharma, and Kevin Duda. 2022. Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor. In *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, Vol. 65. IEEE, Online due to COVID-19, 44–46. https://doi.org/10.1109/ISSCC42614.2022.9731107

[31] DPDK Project. Accessed in 2025. DPDK Programmer's Guide – Service Cores. https://doc.dpdk.org/guides/prog_guide/service_cores.html

[32] Ravindra P Saraf, Rahul Pal, and Ashok Jagannathan. U.S. Patent 8862828B2 Oct. 2014. Sub-NUMA clustering.

[33] Sarabjeet Singh and Manu Awasthi. 2019. Memory Centric Characterization and Analysis of SPEC CPU2017 Suite. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering* (Mumbai, India) *(ICPE '19)*. Association for Computing Machinery, New York, NY, USA, 285–292. https://doi.org/10.1145/3297663.3310311

[34] Igor Smolyar, Alex Markuze, Boris Pismenny, Haggai Eran, Gerd Zellweger, Austin Bolen, Liran Liss, Adam Morrison, and Dan Tsafrir. 2020. IOctopus: Outsmarting Nonuniform DMA. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 101–115. https://doi.org/10.1145/3373376.3378509

[35] Wei Su, Abhishek Dhanotia, Carlos Torres, Jayneel Gandhi, Neha Gholkar, Shobhit Kanaujia, Maxim Naumov, Kalyan Subramanian, Valentin Andrei, Yifan Yuan, and Chunqiang Tang. 2025. DCPerf: An Open-Source, Battle-Tested Performance Benchmark Suite for Datacenter Workloads. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1717–1730. https://doi.org/10.1145/3695053.3731411

[36] The OpenSSL Project. Accessed in 2025. OpenSSL: The Open Source toolkit for SSL/TLS. www.openssl.org.

[37] Amin Tootoonchian, Aurojit Panda, Chang Lan, Melvin Walls, Katerina Argyraki, Sylvia Ratnasamy, and Scott Shenker. 2018. {ResQ}:

Enabling {SLOs} in network function virtualization. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton WA USA, 283–297. https://doi.org/10.5555/3307441.3307466

[38] Marina Vemmou, Albert Cho, and Alexandros Daglis. 2023. Patching up Network Data Leaks with Sweeper. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture* (Chicago, Illinois, USA) *(MICRO '22)*. IEEE Press, 464–479. https://doi.org/10.1109/MICRO56248.2022.00041

[39] Krishnaswamy Viswanathan. 2024. Intel® Memory Latency Checker v3.11. https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html

[40] Midhul Vuppalapati, Saksham Agarwal, Henry Schuh, Baris Kasikci, Arvind Krishnamurthy, and Rachit Agarwal. 2024. Understanding the Host Network. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 581–594. https://doi.org/10.1145/3651890.3672271

[41] Minhu Wang, Mingwei Xu, and Jianping Wu. 2022. Understanding I/O Direct Cache Access Performance for End Host Networking. *Proc. ACM Meas. Anal. Comput. Syst.* 6, 1, Article 22 (Feb. 2022), 37 pages. https://doi.org/10.1145/3508042

[42] Mengjia Yan, Read Sprabery, Bhargava Gopireddy, Christopher Fletcher, Roy Campbell, and Josep Torrellas. 2019. Attack directories, not caches: Side channel attacks in a non-inclusive world. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, IEEE, San Francisco, CA, USA, 888–904. https://doi.org/10.1109/SP.2019.00004

[43] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. 2017. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference* (London, United Kingdom) *(IMC '17)*. Association for Computing Machinery, New York, NY, USA, 78–85. https://doi.org/10.1145/3131365.3131375